

Material instrucional

Tutorial completo

CopaFigurinhas com agente analisador de CI/CD

Aplicação full stack, qualidade automatizada, Pipeline Guardian, investigação de falhas, deploy assistido e demonstração guiada

Este tutorial foi estruturado do zero para o repositório pipeline-guardian-agent-demo. Você construirá uma aplicação de álbum de figurinhas e, em seguida, adicionará qualidade automatizada, um pipeline no GitHub Actions e o agente Pipeline Guardian. Ao longo da prática, o agente aprenderá a observar resultados do ESLint, testes e build, analisar logs e alterações de uma Pull Request, classificar falhas, sugerir causas prováveis, indicar impacto e próximos passos e produzir diagnósticos rastreáveis.

Ao longo do tutorial, você irá:

- criar a aplicação CopaFigurinhas com Node.js, Express, React e Vite
- configurar ESLint, Vitest, Supertest e React Testing Library
- implementar o Pipeline Guardian com fallback determinístico e saída estruturada
- integrar o agente ao GitHub Actions usando logs, artefatos e diff da Pull Request
- evoluir o analisador para um agente investigativo com ferramentas e loop controlado
- simular deploy assistido com gates, política, aprovação humana e rollback planejado
- reproduzir uma falha de regra de negócio e interpretar o diagnóstico dos slides 15 e 16
- construir a demonstração guiada do slide 19 em um notebook reutilizando o agente existente

Linhagem da prática

O workflow executa tarefas determinísticas. O agente escolhe como investigar e recomenda uma ação. A política define o que está autorizado. Ações críticas permanecem condicionadas a permissões e aprovação humana.

1. Como o tutorial está organizado

Você não precisará programar em todos os slides. A aplicação e o agente aparecem em pontos específicos, e cada etapa acrescenta uma nova capacidade.

Slide	Parte prática
1-7	Conceitos e visão geral
8	Agente analisa PR, ESLint, testes e logs
9	Agente analisa build e prepara informações de release
10	Agente recomenda deploy, mas uma política e um humano controlam produção
11	Diagnóstico é publicado no GitHub, ainda sem Slack ou Discord
12-14	Explicação da arquitetura, entrada, processamento e logs
15	Laboratório com diferentes categorias de falha
16	Transformação do log bruto em resposta estruturada
17-18	Limites, segredos e ações críticas
19	Desafio separado em notebook

A progressão prática será:

Aplicação funcionando

```
↓
ESLint + testes + build
↓
Pipeline GitHub Actions
↓
Coleta de logs e diff
↓
Agente interpreta os sinais
↓
Política determinística controla a decisão
↓
Diagnóstico no GitHub
```

2. O que será a aplicação Copa Figurinhas

A aplicação permitirá controlar um pequeno álbum de figurinhas de jogadores.

Funcionalidades

O usuário poderá:

- cadastrar uma figurinha
- remover uma figurinha
- visualizar os jogadores cadastrados
- adicionar uma cópia repetida
- remover uma cópia
- identificar figurinhas faltantes
- identificar figurinhas repetidas

- pesquisar por jogador
- filtrar por país e situação
- visualizar o progresso da coleção
- gerar um relatório

copiar ou compartilhar o relatório.

Regras de negócio

Cada jogador possui apenas um registro no álbum.

A quantidade representa o estado da figurinha:

quantity = 0 → figurinha faltante

quantity = 1 → figurinha obtida

quantity > 1 → figurinha obtida com repetidas

As cópias repetidas são calculadas assim:

- `const duplicateCopies = Math.max(quantity - 1, 0)`

Exemplo:

Jogador	Quantidade	Situação	Repetidas
Lucas Andrade	0	Faltante	0
Mateo Ruiz	1	Obtida	0
Noah Martins	3	Obtida e repetida	2

Essa regra será especialmente útil no slide 15, porque podemos introduzir uma falha no cálculo das repetidas e deixar o agente analisar o erro.

3. Modelo de dados

Cada figurinha terá esta estrutura:

```
{
  "id": "stk-001",
  "albumNumber": 10,
  "playerName": "Lucas Andrade",
  "country": "Brasil",
  "countryCode": "BR",
  "position": "forward",
  "quantity": 2,
  "createdAt": "2026-07-13T12:00:00.000Z",
  "updatedAt": "2026-07-13T12:00:00.000Z"
}
```

Regras dos campos

Campo	Regra
albumNumber	Inteiro positivo e único
playerName	Obrigatório, de 3 a 80 caracteres
country	Obrigatório
countryCode	Duas letras maiúsculas

position	goalkeeper, defender, midfielder ou forward
quantity	Inteiro maior ou igual a zero

4. Endpoints do backend

```
GET    /api/health
GET    /api/stickers
GET    /api/stickers/:id
POST   /api/stickers
PATCH /api/stickers/:id/quantity
DELETE /api/stickers/:id
GET    /api/report
```

Cadastro

```
POST /api/stickers
Content-Type: application/json
{
  "albumNumber": 10,
  "playerName": "Lucas Andrade",
  "country": "Brasil",
  "countryCode": "BR",
  "position": "forward",
  "quantity": 1
}
```

Adicionar uma cópia

```
PATCH /api/stickers/stk-001/quantity
Content-Type: application/json
{
  "operation": "increment"
}
```

Remover uma cópia

```
{
  "operation": "decrement"
}
O backend deve rejeitar uma operação que deixe a quantidade negativa.
```

5. Design da interface

A aplicação terá um visual esportivo, mas sem usar símbolos, marcas oficiais ou fotografias reais.

Paleta

```
:root {
  --navy-900: #071a2b;
  --navy-700: #12344d;
  --green-600: #07805b;
  --green-400: #2fbf71;
  --gold-500: #e9b949;
  --red-500: #d64545;
  --cream-100: #f7f5ef;
  --white: #ffffff;
  --gray-700: #52606d;
  --gray-200: #d9e2ec;
}
```

Estrutura visual

 CopaFigurinhas		Compartilhar relatório	
Organize sua coleção e encontre suas repetidas			
Progresso 72% Obtidas 18 Faltantes 7 Repetidas 9			
Buscar...	País ▼	Situação ▼	+ Nova figurinha
[Figurinha]	[Figurinha]	[Figurinha]	[Figurinha]
[Figurinha]	[Figurinha]	[Figurinha]	[Figurinha]

Card de figurinha

#010	BRASIL
LA	
Lucas Andrade	
Atacante	
Quantidade: 3	
Repetidas: 2	
[-] 3 [+]	
Excluir	

Estados visuais:

- faltante: borda vermelha e opacidade reduzida
 - obtida: borda verde
 - repetida: borda e selo dourados
 - erro: mensagem clara acima do grid
- carregamento: skeleton simples.

6. Arquitetura do projeto

pipeline-guardian-agent-demo/

backend/
src/
app.js
server.js
controllers/
report-controller.js
sticker-controller.js
data/
initial-stickers.js
middleware/
error-handler.js
not-found.js
request-id.js
routes/
report-routes.js
sticker-routes.js
schemas/
sticker-schema.js
services/
report-service.js
sticker-service.js
tests/

```
├── report.test.js
├── stickers.test.js
├── eslint.config.js
├── package.json
├── .env.example
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── CollectionDashboard.jsx
│   │   │   ├── Header.jsx
│   │   │   ├── ReportDialog.jsx
│   │   │   ├── StickerCard.jsx
│   │   │   ├── StickerFilters.jsx
│   │   │   ├── StickerForm.jsx
│   │   │   └── StickerGrid.jsx
│   │   ├── services/
│   │   │   └── sticker-api.js
│   │   ├── utils/
│   │   │   └── share-report.js
│   │   ├── App.css
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── tests/
│   ├── eslint.config.js
│   ├── vite.config.js
│   ├── package.json
│   └── .env.example
├── automation/
│   ├── src/
│   │   ├── analyze-pipeline.mjs
│   │   ├── collect-context.mjs
│   │   ├── deterministic-classifier.mjs
│   │   ├── deploy-policy.mjs
│   │   ├── redact-secrets.mjs
│   │   ├── render-report.mjs
│   │   ├── simulate-failure.mjs
│   │   └── upsert-pr-comment.mjs
│   ├── prompts/
│   │   └── pipeline-analysis.md
│   ├── schemas/
│   │   └── diagnosis-schema.mjs
│   ├── fixtures/
│   │   └── logs/
│   ├── tests/
│   ├── package.json
│   └── .env.example
├── reports/
│   └── .gitkeep
├── docs/
│   ├── ARCHITECTURE.md
│   ├── DEMO-GUIDE.md
│   └── GITHUB-SETUP.md
├── .github/
│   ├── workflows/
│   │   ├── ci.yml
│   │   ├── deploy-assisted.yml
│   │   ├── failure-lab.yml
│   │   └── release.yml
│   └── pull_request_template.md
├── CLAUDE.md
├── package.json
├── package-lock.json
├── .gitignore
└── README.md
```

7. Preparação do ambiente

A documentação atual do Vite utiliza npm create vite@latest e exige Node.js 20.19+ ou 22.12+. Para este laboratório, use Node 22.12 ou uma versão 22 mais recente. O ESLint atual usa arquivos de configuração plana, como eslint.config.js. O Vitest pode ser instalado como dependência de desenvolvimento e requer Node 20 ou superior.

7.1 Clonar o repositório

```
git clone git@github.com:SEU_USUARIO/pipeline-guardian-agent-demo.git
cd pipeline-guardian-agent-demo
Confirme:
git remote -v
git status
git branch --show-current
```

7.2 Configurar Node

```
nvm install 22
nvm use 22

node --version
npm --version
A versão do Node deve ser pelo menos:
v22.12.0
```

7.3 Criar a primeira branch

```
git checkout -b feat/copa-figurinhas-base
```

7.4 Abrir o Claude Code

claude

O Claude Code consegue ler o repositório, editar múltiplos arquivos e executar comandos. Por isso, cada prompt abaixo solicita implementação e validação, mas impede commit e push automáticos.

8. Etapa 1 - Criar a aplicação CopaFigurinhas

Onde esta etapa acontece

- branch feat/copa-figurinhas-base
- Claude Code

execução local.

Prompt 1 - aplicação completa

Cole o prompt abaixo no Claude Code:

```
Você é um engenheiro de software especialista em Node.js, Express,
React, Vite, APIs REST e design de interfaces educacionais.

Trabalhe no repositório atual pipeline-guardian-agent-demo.

Antes de implementar:
1. inspecione todos os arquivos existentes;
2. apresente um plano breve;
3. implemente o plano depois da apresentação;
4. não faça commit;
5. não faça push;
6. não crie Pull Request;
7. não implemente agente de IA ou GitHub Actions nesta etapa.
```

OBJETIVO

Criar uma aplicação full stack chamada CopaFigurinhas.

A aplicação será usada posteriormente em uma aula de CI/CD e agentes, portanto precisa ser simples de entender, organizada e suficientemente completa para permitir testes, lint e build.

ARQUITETURA

Use um monorepo com npm workspaces:

- backend
- frontend

Use JavaScript com ES Modules. Não use TypeScript.

Crie um package.json na raiz com:

- private: true
- workspaces para backend e frontend
- scripts para dev, lint, test, build e ci
- concurrently para executar backend e frontend juntos

BACKEND

Use:

- Node.js
- Express
- CORS
- Zod
- dados armazenados em memória

Separe app.js de server.js para permitir testes.

Crie os endpoints:

- GET /api/health
- GET /api/stickers
- GET /api/stickers/:id
- POST /api/stickers
- PATCH /api/stickers/:id/quantity
- DELETE /api/stickers/:id
- GET /api/report

MODELO

Cada figurinha deve possuir:

- id
- albumNumber
- playerName
- country
- countryCode
- position
- quantity
- createdAt
- updatedAt

REGRAS

- albumNumber deve ser inteiro positivo e único
- playerName deve ter entre 3 e 80 caracteres
- country é obrigatório
- countryCode deve possuir duas letras
- position deve ser:
goalkeeper, defender, midfielder ou forward
- quantity deve ser inteiro maior ou igual a zero

- quantity igual a zero significa faltante
- quantity igual a um significa obtida sem repetidas
- quantity maior que um significa obtida com repetidas
- duplicateCopies deve ser Math.max(quantity - 1, 0)
- decremento nunca pode produzir quantidade negativa
- item inexistente retorna 404
- entrada inválida retorna 400
- criação válida retorna 201
- exclusão válida retorna 204

PATCH DE QUANTIDADE

O endpoint deve aceitar:

```
{  
  "operation": "increment"  
}
```

ou:

```
{  
  "operation": "decrement"  
}
```

RELATÓRIO

GET /api/report deve retornar:

- totalRegistered
- obtained
- missing
- duplicateCopies
- completionPercentage
- byCountry
- missingStickers
- duplicateStickers

completionPercentage deve representar:

$\text{obtained} / \text{totalRegistered} * 100$

Arredonde para inteiro.

DADOS INICIAIS

Crie doze figurinhas fictícias.

Distribua entre pelo menos quatro países e quatro posições.

Inclua exemplos com:

- quantity 0
- quantity 1
- quantity 2
- quantity 3

Não use APIs externas e não busque imagens.

BACKEND - QUALIDADE

Inclua:

- middleware de requestId
- resposta de erro padronizada
- middleware 404
- logs simples em JSON
- nenhuma impressão de Authorization ou cookies
- .env.example
- porta padrão 3001

FRONTEND

Use React com Vite.

A interface deve ter:

1. cabeçalho com:
 - nome CopaFigurinhas
 - subtítulo
 - botão Compartilhar relatório
2. painel de indicadores:
 - percentual de conclusão
 - obtidas
 - faltantes
 - repetidas
3. filtros:
 - busca por jogador
 - país
 - situação: todas, obtidas, faltantes, repetidas
4. formulário:
 - número no álbum
 - nome
 - país
 - código do país
 - posição
 - quantidade inicial
5. grid de figurinhas:
 - número
 - iniciais do jogador
 - nome
 - país
 - posição
 - quantidade
 - repetidas
 - botão incrementar
 - botão decrementar
 - botão excluir
6. relatório:
 - resumo
 - faltantes
 - repetidas
 - botão copiar
 - botão compartilhar

COMPARTILHAMENTO

Use navigator.share quando disponível.

Use navigator.clipboard.writeText como fallback.

Não use bibliotecas externas para compartilhamento.

DESIGN

Visual esportivo, moderno e profissional.

Use:

- azul profundo #071A2B
- verde #07805B
- verde claro #2FBF71
- dourado #E9B949
- vermelho #D64545
- fundo #F7F5EF

```
Não use framework CSS.

Não use imagens externas.

Use iniciais, badges, ícones simples e CSS.

Crie interface responsiva para desktop e mobile.

CONFIGURAÇÃO FRONTEND

Use VITE_API_URL.

Crie frontend/.env.example com:

VITE_API_URL=http://localhost:3001

Configure proxy de desenvolvimento se for útil.

DOCUMENTAÇÃO

Crie README.md contendo:

- objetivo
- arquitetura
- tecnologias
- instalação
- execução
- endpoints
- regras de negócio
- scripts
- aviso de armazenamento em memória

Crie CLAUDE.md contendo:

- decisões arquiteturais
- comandos principais
- restrições
- padrão de nomes
- regra para não introduzir banco de dados, autenticação ou Docker

VALIDAÇÃO

Ao terminar:

1. execute npm install na raiz;
2. execute a aplicação;
3. valide o health check;
4. valide o build do frontend;
5. corrija erros encontrados;
6. apresente a árvore de arquivos criada;
7. liste os comandos executados;
8. não faça commit ou push.
```

8.1 Validar localmente

Depois que o Claude Code finalizar:

```
npm install
npm run dev
Acesse:
Frontend: http://localhost:5173
Backend: http://localhost:3001
Health: http://localhost:3001/api/health
Teste pelo terminal:
curl http://localhost:3001/api/health
curl http://localhost:3001/api/stickers
curl http://localhost:3001/api/report
Teste um cadastro:
curl -X POST http://localhost:3001/api/stickers \
  -H "Content-Type: application/json" \
```

```
-d '{
  "albumNumber": 50,
  "playerName": "Rafael Monteiro",
  "country": "Brasil",
  "countryCode": "BR",
  "position": "midfielder",
  "quantity": 1
}'
```

8.2 Versionar a aplicação-base

```
git status
git add .
git commit -m "feat: create CopaFigurinhas full stack application"
git push -u origin feat/copa-figurinhas-base
Abra uma Pull Request para main.
Depois de revisar a aplicação, faça o merge.
```

9. Etapa 2 - Adicionar ESLint e testes

Crie uma nova branch:

```
git checkout main
git pull
git checkout -b feat/quality-and-tests
```

Prompt 2 - qualidade automatizada

Você é um especialista em testes JavaScript, ESLint, Vitest, Supertest e React Testing Library.

Analise o repositório atual antes de alterar arquivos.

Implemente uma camada completa de qualidade para a aplicação CopaFigurinhas.

Não faça commit.

Não faça push.

Não altere as regras de negócio existentes.

Não implemente GitHub Actions ou agente de IA nesta etapa.

ESLINT

Use flat config com eslint.config.js.

Configure backend e frontend.

Backend:

- eslint recommended
- ambiente Node
- ES Modules
- no-unused-vars
- no-undef
- eqeqeq
- no-unreachable
- prefer-const

Frontend:

- eslint recommended
- plugin react-hooks
- plugin react-refresh
- ambiente browser
- JSX habilitado
- no-unused-vars
- no-undef
- eqeqeq

- no-unreachable

Ignore:

- node_modules
- dist
- coverage
- reports
- artefatos gerados

TESTES DO BACKEND

Use Vitest e Supertest.

Crie testes independentes e previsíveis para:

- GET /api/health
- GET /api/stickers
- GET /api/stickers/:id
- POST válido
- nome inválido
- posição inválida
- quantidade negativa
- albumNumber duplicado
- incremento
- decremento
- tentativa de quantidade negativa
- exclusão
- recurso inexistente
- relatório
- cálculo de duplicateCopies
- percentual de conclusão
- agrupamento por país

Garanta reset do armazenamento em memória entre os testes.

TESTES DO FRONTEND

Use:

- Vitest
- jsdom
- React Testing Library
- user-event

Teste:

- título CopaFigurinhas
- painel de indicadores
- carregamento inicial
- cadastro de figurinha
- incremento
- decremento
- filtro de faltantes
- filtro de repetidas
- busca por jogador
- erro da API
- abertura do relatório
- cópia do relatório

Mocke a API nos testes.

SCRIPTS

Na raiz, disponibilize:

- npm run lint
- npm run test
- npm run test:backend
- npm run test:frontend

```
- npm run build
- npm run ci

npm run ci deve executar:

1. lint
2. testes
3. build

VALIDAÇÃO

Execute:

- npm run lint
- npm run test
- npm run build
- npm run ci

Corrija qualquer falha.

Ao final, apresente:

- quantidade de testes
- arquivos adicionados
- comandos executados
- resultado de lint, testes e build

Não faça commit ou push.
```

9.1 Validar

```
npm run lint
npm run test
npm run build
npm run ci
```

9.2 Versionar

```
git add .
git commit -m "test: add lint and automated test suites"
git push -u origin feat/quality-and-tests
Abra uma PR e faça o merge depois que os testes passarem.
```

10. Etapa 3 - Criar o Pipeline Guardian

O Pipeline Guardian não será um agente com autonomia irrestrita.

Ele terá um fluxo controlado:

Coleta ferramentas e evidências

```
↓
Remove dados sensíveis
↓
Interpreta logs e contexto
↓
Gera diagnóstico estruturado
↓
Política determinística revisa a decisão
↓
Publica JSON e Markdown
```

Responsabilidade da IA

A IA pode:

- selecionar sinais relevantes
- classificar a falha

- sugerir uma causa provável
- explicar impacto

sugerir próximos passos.

Responsabilidade da política fixa

A política deve decidir:

- lint falhou → bloqueado
- teste falhou → bloqueado
- build falhou → bloqueado
- segredo detectado → bloqueado e risco alto
- contexto insuficiente → bloqueado
- staging com tudo aprovado → elegível

produção → sempre exige aprovação humana.

A API da OpenAI oferece saída estruturada com schemas Zod no SDK JavaScript, o que permite validar o objeto retornado pelo modelo antes de utilizá-lo.

10.1 Criar branch

```
git checkout main
git pull
git checkout -b feat/pipeline-guardian
```

Prompt 3 - agente analisador

Você é um especialista em agentes de IA, Node.js, análise de CI/CD, OpenAI API, Zod, segurança de logs e fallback determinístico.

Analise o repositório CopaFigurinhas antes de modificar arquivos.

Crie um novo workspace chamado automation.

Não use LangGraph.
 Não use Python.
 Não use Slack ou Discord.
 Não execute deploy real.
 Não faça commit ou push.

OBJETIVO

Implementar um agente chamado Pipeline Guardian.

Ele deve receber contexto de pipeline, logs técnicos e diff de uma Pull Request, produzindo um diagnóstico estruturado e uma versão Markdown.

O projeto deve funcionar com ou sem OPENAI_API_KEY.

ESTRUTURA

Crie:

```
automation/
├── src/
│   ├── analyze-pipeline.mjs
│   ├── collect-context.mjs
│   ├── deterministic-classifier.mjs
│   ├── deploy-policy.mjs
│   ├── redact-secrets.mjs
│   ├── render-report.mjs
│   ├── simulate-failure.mjs
│   └── upsert-pr-comment.mjs
├── prompts/
│   └── pipeline-analysis.md
└── schemas/
```

```
├── diagnosis-schema.mjs
├── fixtures/
│   └── logs/
├── tests/
├── package.json
└── .env.example
```

DEPENDÊNCIAS

Use:

- openai
- zod
- dotenv

SCHEMA

Crie um schema Zod com:

- analysisId
- requestId
- repository
- branch
- commitSha
- pipelineStatus:
 - success, failed ou partial
- summary
- signal
- failureType:
 - lint, test, dependency, build, environment, permission, security ou unknown
- probableCause
- evidence:
 - array de objetos com source e excerpt
- impact
- riskLevel:
 - low, medium ou high
- confidence:
 - low, medium ou high
- nextSteps:
 - array de strings
- deployDecision:
 - eligible_for_staging, blocked ou requires_human_approval
- requiresHumanApproval
- limitations:
 - array de strings
- usedFallback
- generatedAt

FERRAMENTAS INTERNAS

Implemente funções com responsabilidades separadas:

- readPipelineMetadata
- readCommandLogs
- readPullRequestDiff
- inspectCommandResults
- scanForSensitiveData
- buildEvidenceList

Essas funções devem ser utilizadas pelo agente como ferramentas de coleta.

REDAÇÃO DE SEGREDOS

Antes de enviar qualquer conteúdo para o modelo, remova ou mascare:

- OPENAI_API_KEY
- Authorization Bearer
- tokens iniciados por ghp_
- tokens iniciados por github_pat_

- valores iniciados por sk-
- variáveis com nome PASSWORD, SECRET ou TOKEN
- credenciais em URLs
- cookies

Substitua valores por:

[REDACTED]

Nunca inclua o valor original no diagnóstico.

CLASSIFICADOR DETERMINÍSTICO

Crie um classificador baseado em padrões para os tipos:

lint:

- no-unused-vars
- no-undef
- ESLint

test:

- AssertionError
- expected
- received
- FAIL

dependency:

- ERR_MODULE_NOT_FOUND
- Cannot find package
- module not found

build:

- Could not resolve
- build failed
- RollupError

environment:

- missing env
- is required but was not provided
- undefined environment variable

permission:

- EACCES
- permission denied
- 403 Forbidden

security:

- token detectado
- segredo detectado
- credencial detectada

unknown:

- nenhum padrão confiável

OPENAI

Quando OPENAI_API_KEY e OPENAI_MODEL existirem:

- use a Responses API;
- use saída estruturada validada com Zod;
- envie apenas dados já mascarados;
- limite o contexto aos trechos relevantes;
- não permita que o modelo defina sozinho a decisão final de deploy.

Quando não houver chave, ocorrer erro ou a saída for inválida:

- use o classificador determinístico;
- marque usedFallback como true;
- ainda gere saída válida.

PROMPT DO AGENTE

O prompt deve instruir o modelo a:

- distinguir evidência de hipótese;
- não inventar arquivos ou comandos;
- informar limitações;
- classificar confiança;
- não recomendar merge automático;
- não recomendar deploy automático em produção;
- não expor segredos;
- produzir apenas o objeto solicitado.

POLÍTICA DE DEPLOY

Aplique a política depois da resposta do modelo.

Regras:

- lint falhou: blocked
- testes falharam: blocked
- build falhou: blocked
- security detectado: blocked e risco high
- permission: blocked
- confidence low: blocked
- limitations relevantes: blocked
- production: requires_human_approval
- rollback de production: requires_human_approval
- staging e todos os gates aprovados: eligible_for_staging

A política deve sobrescrever qualquer recomendação insegura do modelo.

SAÍDAS

Gerar:

- reports/diagnosis.json
- reports/diagnosis.md

O Markdown deve apresentar:

- resumo
- sinal
- tipo
- causa provável
- evidências
- impacto
- risco
- confiança
- próximos passos
- decisão
- aprovação humana
- limitações
- indicação de fallback

FIXTURES

Crie logs de exemplo para:

- lint
- test
- dependency
- build
- environment
- permission
- security
- unknown

TESTES

Teste:

- cada classificação
- redação de segredos
- schema
- fallback sem chave
- política de staging
- política de production
- bloqueio de testes
- bloqueio de build
- bloqueio por segurança
- confiança baixa
- geração Markdown

SCRIPTS

Na raiz:

- npm run agent:analyze
- npm run agent:fixture -- <scenario>

VALIDAÇÃO

Execute todos os testes e o lint.

Gere um diagnóstico usando o fixture test.

Mostre o JSON e o Markdown produzidos.

Não faça commit ou push.

10.2 Ambiente local

Crie:

```
cp automation/.env.example automation/.env
Conteúdo:
OPENAI_API_KEY=
OPENAI_MODEL=
AGENT_MODE=hybrid
Sem chave, o agente deve continuar funcionando.
```

10.3 Testar

```
npm run agent:fixture -- test
npm run agent:fixture -- dependency
npm run agent:fixture -- security
Verifique:
reports/diagnosis.json
reports/diagnosis.md
```

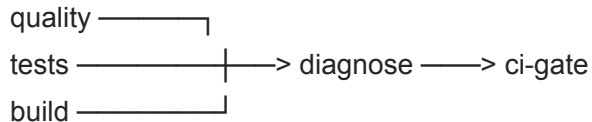
10.4 Versionar

```
git add .
git commit -m "feat: add Pipeline Guardian analysis agent"
git push -u origin feat/pipeline-guardian
Abra a PR, revise e faça o merge.
```

11. Etapa 4 - Criar o pipeline GitHub Actions

A documentação atual do GitHub utiliza actions/checkout@v6, actions/setup-node@v4, actions/upload-artifact@v4 e actions/download-artifact@v5. Os artefatos podem ser usados para transportar logs e resultados entre jobs.

Arquitetura do workflow



Cada job técnico:

- executa o comando
- salva o log
- publica o log como artefato

mantém o resultado real de sucesso ou falha.

O diagnose roda mesmo quando outro job falha.

11.1 Criar branch

```
git checkout main
git pull
git checkout -b feat/github-actions-ci
```

Prompt 4 - pipeline de CI com diagnóstico

Você é um especialista em GitHub Actions, Node.js, artefatos, segurança de workflows e pipelines CI/CD.

Analise toda a aplicação CopaFigurinhas e o workspace automation.

Implemente `.github/workflows/ci.yml`.

Não faça deploy.

Não faça commit.

Não faça push.

EVENTOS

Executar em:

- pull_request para main
- push para main

PERMISSÕES

Use permissões mínimas:

- contents: read
- pull-requests: write

JOBS

Crie:

1. quality
2. tests
3. build
4. diagnose
5. ci-gate

Use:

- ubuntu-latest
- actions/checkout@v6
- actions/setup-node@v4
- Node 22
- cache npm
- npm ci
- actions/upload-artifact@v4

- actions/download-artifact@v5

QUALITY

Execute npm run lint.

Salve a saída em:

reports/lint.log

Use tee e preserve o exit code.

Mesmo quando o lint falhar:

- faça upload do log;
- finalize o job como falha depois do upload.

TESTS

Execute npm run test.

Salve:

reports/tests.log

Faça upload mesmo em falha.

BUILD

Execute npm run build.

Salve:

reports/build.log

Faça upload mesmo em falha.

DIAGNOSE

Use:

needs:

- quality
- tests
- build

E:

if: always()

O job deve:

1. fazer checkout;
2. instalar dependências;
3. baixar os artefatos;
4. consolidar logs em reports/input;
5. coletar:
 - repository
 - branch
 - commit SHA
 - run id
 - evento
 - resultados de quality, tests e build
6. em Pull Request, obter o diff da PR;
7. executar o Pipeline Guardian;
8. gerar:
 - reports/diagnosis.json
 - reports/diagnosis.md
9. adicionar diagnosis.md a GITHUB_STEP_SUMMARY;
10. publicar os dois arquivos como artefato;

11. criar ou atualizar um comentário na Pull Request.

O comentário não pode ser duplicado a cada execução.

Use um marcador HTML como:

```
<!-- pipeline-guardian-diagnosis -->
```

Implemente ou utilize automation/src/upsert-pr-comment.mjs.

Passe GITHUB_TOKEN apenas por variável de ambiente.

Nunca escreva o token no log.

OPENAI

Passe:

- OPENAI_API_KEY usando secrets.OPENAI_API_KEY
- OPENAI_MODEL usando vars.OPENAI_MODEL

O workflow deve funcionar sem esses valores.

CI-GATE

Use if: always().

O job deve falhar se qualquer um destes jobs falhar:

- quality
- tests
- build

O diagnóstico não deve transformar um pipeline quebrado em sucesso.

ARTEFATOS

Publique:

- lint.log
- tests.log
- build.log
- diagnosis.json
- diagnosis.md
- pr.diff, quando existir

DOCUMENTAÇÃO

Atualize docs/GITHUB-SETUP.md com:

- secret OPENAI_API_KEY
- variable OPENAI_MODEL
- Workflow permissions
- explicação dos jobs
- localização dos artefatos
- localização do Job Summary
- comportamento com fallback

VALIDAÇÃO

Verifique a sintaxe YAML.

Execute localmente:

- npm run lint
- npm run test
- npm run build
- npm run agent:fixture -- test

Não faça commit ou push.

11.2 Versionar

```
git add .
git commit -m "ci: add GitHub Actions pipeline diagnosis"
git push -u origin feat/github-actions-ci
Abra uma PR. Nesse momento, o próprio pipeline deve começar a funcionar.
```

12. Configuração no GitHub

12.1 Secret da OpenAI

No repositório:

Settings

```
→ Secrets and variables
→ Actions
→ Secrets
→ New repository secret
Nome:
OPENAI_API_KEY
```

12.2 Variável do modelo

Settings

```
→ Secrets and variables
→ Actions
→ Variables
→ New repository variable
Nome:
OPENAI_MODEL
Use um modelo disponível na sua conta ou no ambiente fornecido para a prática.
```

12.3 Permissão do workflow

Settings

```
→ Actions
→ General
→ Workflow permissions
Permita que o workflow consiga publicar comentários em Pull Requests.
O próprio YAML continuará declarando permissões mínimas.
```

12.4 Proteção da main

Settings

```
→ Rules
→ Rulesets
Crie uma regra para main exigindo:
```

- Pull Request antes de merge
- branch atualizada
- aprovação
- checks obrigatórios

bloqueio de push direto.

Checks recomendados:

Quality / ESLint

Tests / Vitest

Build / Vite

13. Antes do Slide 8 - Evoluir o Pipeline Guardian para um agente investigativo

13.1 O que muda

Implementação anterior

GitHub Actions

```
→ baixa todos os logs
→ entrega todo o conteúdo ao analisador
→ classificação
→ diagnóstico
Nova implementação
GitHub Actions
→ informa resultados e artefatos disponíveis
→ Pipeline Guardian cria um plano
→ escolhe qual evidência consultar
→ executa uma ferramenta
→ avalia o contexto
→ continua, encerra ou escala
→ política controla a ação final
O GitHub Actions continua sendo o executor e o ambiente controlado. A autonomia está dentro do
job:
Diagnose (Pipeline Guardian)
A distinção que você deve observar é:
Workflow:
define quando o processo inicia e quais recursos estão disponíveis.
```

Agente:

decide como investigar, quais ferramentas usar e quando parar.

Política:

define o que pode ser executado com segurança.

13.2 Autonomia permitida

O agente poderá:

- selecionar qual log consultar
- ignorar logs irrelevantes
- consultar o diff da PR
- localizar arquivos alterados
- buscar um trecho de código
- consultar package.json
- verificar se um arquivo importado existe
- procurar indícios de segredo
- definir se as evidências são suficientes
- continuar a investigação
- encerrar com limitação explícita

recomendar bloqueio, staging ou aprovação humana.

O agente não poderá:

- modificar código
- fazer commit
- fazer merge

- alterar secrets
- autorizar produção
- executar rollback real

publicar automaticamente em produção.

13.3 Estado mínimo da investigação

O agente deverá trabalhar com um estado semelhante a:

```
{
  requestId,
  goal,
  pipelineContext,
  availableArtifacts,
  availableTools,
```

```
  attempts: 0,
  maxAttempts: 5,
  selectedTool: null,
  observations: [],
  evidence: [],
  hypotheses: [],
  confidence: "low",
  diagnosis: null,
  stopReason: null,
  agentRecommendation: null,
  policyDecision: null,
  investigationTrace: []
}
```

13.4 Ferramentas do agente

Crie ferramentas somente de leitura:

```
inspect_job_results
read_pipeline_log
list_changed_files
read_pull_request_diff
read_source_excerpt
search_repository
check_path_exists
inspect_package_manifest
scan_sensitive_data
```

Exemplo:

```
inspect_job_results
```

```
→ quality falhou
→ tests passou
→ build passou
```

Decisão do agente:

consultar somente lint.log

Depois:

`read_pipeline_log("lint")`

→ `reportPreview` foi declarada, mas não utilizada

Decisão do agente:

consultar o diff para verificar se a variável entrou nesta PR

13.5 Loop controlado

O fluxo interno será:

`observar_estado`

```
→ planejar_próximo_passo
→ selecionar_ferramenta
→ executar_ferramenta
→ registrar_observação
→ avaliar_contexto
    ├── contexto suficiente → gerar diagnóstico
    ├── contexto insuficiente → nova ferramenta
    ├── limite atingido → fallback ou escalonamento
    └── risco crítico → interromper
→ aplicar_política
→ publicar_resposta
```

Limites

Configure:

máximo de chamadas de ferramenta: 5

tempo máximo de investigação: 30 segundos

tamanho máximo por observação: 6.000 caracteres

tamanho máximo total de contexto: 20.000 caracteres

Critérios de parada

O agente deve encerrar quando:

- houver evidência suficiente para uma hipótese
- a confiança estiver alta
- o risco exigir bloqueio imediato
- não houver mais ferramentas úteis
- o limite de tentativas for atingido

faltar contexto relevante.

13.6 Criar a branch

Essa evolução precisa entrar na main antes das demonstrações.

```
git switch main
git pull origin main
git switch -c feat/agent-ic-investigation-loop
```

13.7 Claude Code

Use uma sessão limpa:

```
/clear
/model opus
/effort high
```

13.8 Prompt - transformar o analisador em agente investigativo

Você é um especialista em agentes de IA, Node.js, pipelines CI/CD, tool calling, planejamento, segurança e observabilidade.

Trabalhe no repositório atual pipeline-guardian-agent-demo.

Antes de alterar arquivos:

1. inspecione o workspace automation;
2. inspecione .github/workflows/ci.yml;
3. identifique como ci-diagnose.mjs chama atualmente o Pipeline Guardian;
4. apresente um plano resumido;
5. depois implemente o plano;
6. não faça commit;
7. não faça push;
8. não modifique a aplicação CopaFigurinhas;
9. não use LangGraph;
10. não implemente deploy real.

OBJETIVO

Evoluir o Pipeline Guardian de um analisador que recebe todo o contexto pronto para um agente investigativo com autonomia controlada.

O agente deve receber inicialmente apenas:

- resultados dos jobs;
- metadados do pipeline;
- lista de artefatos disponíveis;
- lista de ferramentas disponíveis;
- ambiente solicitado;
- objetivo da investigação.

Ele não deve receber antecipadamente todo o conteúdo de todos os logs.

O agente deverá decidir:

- qual evidência consultar;
- qual ferramenta executar;
- se precisa consultar outra evidência;
- quando existe contexto suficiente;
- quando deve parar;
- quando deve responder com limitação;
- quando deve escalar para aprovação humana.

ARQUITETURA

Adapte a estrutura existente sem duplicar implementações.

Crie ou reorganize módulos equivalentes a:

```
automation/src/
├── agent/
│   ├── investigation-state.mjs
│   ├── planner.mjs
│   ├── investigation-loop.mjs
│   ├── evidence-evaluator.mjs
│   └── stop-criteria.mjs
├── tools/
│   ├── tool-registry.mjs
│   ├── inspect-job-results.mjs
│   ├── read-pipeline-log.mjs
│   ├── list-changed-files.mjs
│   ├── read-pull-request-diff.mjs
│   └── read-source-excerpt.mjs
```

```
| ── search-repository.mjs
| ── check-path-exists.mjs
| ── inspect-package-manifest.mjs
| ── scan-sensitive-data.mjs
```

Adapte os nomes caso a estrutura atual já possua módulos equivalentes.

Não apague funcionalidades que já estejam testadas.

ESTADO

O estado da investigação deve conter:

- requestId
- goal
- pipelineContext
- availableArtifacts
- availableTools
- attempts
- maxAttempts
- selectedTool
- observations
- evidence
- hypotheses
- confidence
- diagnosis
- stopReason
- agentRecommendation
- policyDecision
- investigationTrace

PLANEJADOR

O planejador deve produzir uma decisão estruturada validada com Zod:

```
{
  "action": "call_tool | finish | escalate",
  "toolName": "string ou null",
  "arguments": {},
  "reason": "string",
  "expectedEvidence": "string",
  "confidence": "low | medium | high"
}
```

Quando OPENAI_API_KEY e OPENAI_MODEL estiverem presentes:

- usar LLM para planejamento;
- usar saída estruturada;
- enviar apenas contexto mascarado;
- não enviar o repositório inteiro;
- não permitir que o modelo execute comandos diretamente.

Quando não houver LLM:

- utilizar planejador determinístico;
- manter o mesmo contrato;
- marcar plannerMode como deterministic;
- manter o laboratório funcional.

FERRAMENTAS

Todas as ferramentas devem ser somente de leitura.

Implemente:

1. inspect_job_results
Entrada:
 - resultados de quality, tests e build
2. read_pipeline_log

- Entrada:
- tipo: lint, test ou build
 - limite de caracteres
3. list_changed_files
- Entrada:
- arquivo pr.diff
4. read_pull_request_diff
- Entrada:
- arquivo opcional
 - limite de caracteres
5. read_source_excerpt
- Entrada:
- caminho permitido
 - linha inicial
 - linha final
6. search_repository
- Entrada:
- texto de busca
 - diretórios permitidos
 - máximo de resultados
7. check_path_exists
- Entrada:
- caminho relativo
8. inspect_package_manifest
- Entrada:
- root, backend, frontend ou automation
9. scan_sensitive_data
- Entrada:
- fonte ou artefato a verificar

SEGURANÇA DAS FERRAMENTAS

- impedir path traversal;
- não permitir caminhos fora do repositório;
- limitar tamanho de leitura;
- nunca retornar .env;
- nunca retornar secrets;
- mascarar tokens antes de armazenar observações;
- não permitir execução de shell pelo planejador;
- não permitir escrita em arquivos da aplicação;
- registrar cada chamada.

LOOP

Implemente:

1. receber estado inicial;
2. chamar planejador;
3. validar decisão;
4. executar ferramenta autorizada;
5. registrar observação;
6. adicionar evidências;
7. atualizar confiança;
8. avaliar critérios de parada;
9. repetir quando necessário;
10. gerar diagnóstico;
11. aplicar deploy-policy.mjs;
12. gerar JSON e Markdown.

LIMITES

- máximo de 5 ferramentas;
- máximo de 30 segundos;

- máximo de 6.000 caracteres por observação;
- máximo de 20.000 caracteres no contexto agregado;
- ferramentas não podem se repetir sem justificativa;
- ação não reconhecida deve gerar fallback;
- erro de ferramenta deve ser registrado, sem encerrar imediatamente;
- três erros de ferramenta devem provocar escalonamento.

CRITÉRIOS DE PARADA

Encerrar quando:

- houver um sinal técnico objetivo;
- existir correlação com diff, código ou configuração;
- confiança for alta;
- risco crítico exigir bloqueio;
- não houver ferramenta útil;
- limite de tentativas for atingido;
- faltar contexto.

RASTREAMENTO

Cada chamada deve gerar uma entrada:

```
{
  "step": 1,
  "decision": "Consultar o log de lint",
  "tool": "read_pipeline_log",
  "inputSummary": "tipo lint",
  "observation": "reportPreview não é utilizada",
  "evidenceAdded": true
}
```

Adicione à saída:

- plannerMode
- toolsUsed
- toolCallCount
- investigationTrace
- stopReason
- agentRecommendation
- policyDecision

A saída Markdown deve possuir a seção:

Caminho da investigação

Exemplo:

1. Observou falha no job Quality.

2. Consultou lint.log.

3. Identificou no-unused-vars.

4. Consultou o diff da PR.

5. Confirmou que a variável entrou nesta alteração.

6. Encerrou com confiança alta.

INTEGRAÇÃO COM CI

Adapte ci-diagnose.mjs.

Ele deve fornecer ao agente:

- - status dos jobs
 - - caminhos dos artefatos
 - - caminho do pr.diff
 - - metadados do GitHub
- catálogo de ferramentas.

Ele não deve ler e concatenar antecipadamente todos os logs.

Os artefatos podem continuar sendo baixados pelo workflow, mas o agente deve decidir quais arquivos efetivamente ler.

POLÍTICA

A decisão da IA deve permanecer separada da decisão final.

Exemplo:

agentRecommendation:

eligible_for_staging

policyDecision:

blocked

A política sempre prevalece.

TESTES

Crie testes para:

- - limite de chamadas
- - timeout
- - ferramenta não autorizada
- - path traversal
- - mascaramento
- - erro de ferramenta
- - critério de parada
- - contexto insuficiente

- - política prevalecendo sobre o agente
- - planner determinístico
- - planejador LLM mockado
- - geração de trace

- saída Markdown.

Teste os caminhos determinísticos esperados:

LINT:

inspect_job_results

```
→ read_pipeline_log(lint)
→ read_pull_request_diff
→ finish
```

TEST:

inspect_job_results

```
→ read_pipeline_log(test)
→ read_pull_request_diff
→ read_source_excerpt
→ finish
```

BUILD COM IMPORT LOCAL AUSENTE:

inspect_job_results

```
→ read_pipeline_log(build)
→ read_pull_request_diff
→ check_path_exists
→ finish
```

SECURITY:

inspect_job_results

```
→ scan_sensitive_data
→ read_pipeline_log
→ finish
```

UNKNOWN:

inspect_job_results

```
→ read_pipeline_log
→ read_pull_request_diff
→ escalate
```

VALIDAÇÃO

Execute:

- npm run lint
- npm run test
- npm run build
- npm run ci
- fixtures lint, test, build, security e unknown

Mostre:

- - toolsUsed
- - investigationTrace
- - stopReason
- - agentRecommendation
- - policyDecision

- plannerMode.

Não faça commit ou push.

13.9 Validar

```
npm run lint
npm run test
npm run build
npm run ci

npm run agent:fixture -- lint
npm run agent:fixture -- test
npm run agent:fixture -- build
npm run agent:fixture -- unknown
Confirme que os cenários usam caminhos diferentes:
lint:
```

job results → lint.log → diff

test:

job results → tests.log → diff → código

build:

job results → build.log → diff → existência do arquivo

13.10 Versionar e integrar

```
git add .
git commit -m "feat: add controlled investigation loop to Pipeline Guardian"
git push -u origin feat/agent-ic-investigation-loop
Abra uma PR:
base: main
compare: feat/agent-ic-investigation-loop
Título:
feat: add controlled investigation loop to Pipeline Guardian
Faça o merge somente quando:
```

Quality ☒

Tests ☒

Build ☒

Diagnose ☒

CI Gate ☒

Depois:

```
git switch main
git pull origin main
```

14. Slide 10 - Deploy assistido

Gates, aprovação humana e rollback simulado

Deploy assistido não significa permitir que uma LLM publique livremente em produção. Nesta prática, o agente organiza evidências, avalia a prontidão técnica e produz uma recomendação. Depois, uma política determinística revisa essa recomendação antes de qualquer execução.

14.1 O que será demonstrado

Componente	Responsabilidade
GitHub Actions	Executar lint, testes, build e disponibilizar logs e artefatos.
Pipeline Guardian	Interpretar os gates, explicar riscos e recomendar seguir, bloquear ou escalar.
Política de deploy	Aplicar regras fixas e impedir que a recomendação da IA autorize uma ação proibida.
GitHub Environment	Representar o ambiente e, em produção, materializar a aprovação humana quando configurada.

Ponto de atenção

O deploy desta prática é simulado. O resultado é um manifesto que registra o que seria promovido. Nenhuma infraestrutura real é alterada.

14.2 Regras de decisão

Situação observada	Recomendação do agente	Decisão da política
Lint, testes ou build falharam	blocked	blocked
Risco alto ou confiança baixa	escalate ou blocked	blocked
Todos os gates aprovados para staging	eligible_for_staging	eligible_for_staging
Todos os gates aprovados para production	technically_ready	requires_human_approval
Rollback em production	rollback_plan_ready	requires_human_approval

```
graph TD
    A[Solicitação de deploy] --> B[Lint + testes + build]
    B --> C[Pipeline Guardian analisa os resultados]
    C --> D[agentRecommendation]
    D --> E[Política determinística]
    E --> F[policyDecision]
    F --> G[Staging simulado ou aprovação humana]
```

14.3 Criar a branch de implementação

```
git switch main
git pull --ff-only origin main
git switch -c feat/release-and-deploy
```

A branch parte da versão estável da aplicação. As alterações desta etapa são reais e, depois de validadas, devem ser integradas à main por uma Pull Request normal.

14.4 Prompt completo - Release e deploy assistido

Como usar

Cole o prompt abaixo no Claude Code depois de confirmar a branch indicada. O modelo deve analisar o repositório antes de alterar arquivos e não deve executar commit ou push automaticamente.

Você é um especialista em agentes de IA, release engineering, GitHub Actions, segurança de pipelines e deploy assistido.

Trabalhe no repositório atual pipeline-guardian-agent-demo.

Antes de alterar arquivos:

1. confirme que a branch atual é feat/release-and-deploy;
2. inspecione .github/workflows/ci.yml;
3. inspecione o workspace automation e a política existente;
4. identifique os schemas, renderizadores e scripts já disponíveis;
5. apresente um plano curto;
6. implemente somente depois do plano;
7. não faça commit;
8. não faça push;
9. não implemente deploy real;
10. não use provedor de nuvem.

OBJETIVO

Adicionar um fluxo de release e deploy assistido reutilizando o Pipeline Guardian. O agente deve avaliar prontidão técnica. A política determinística deve controlar a decisão final.

CRIE OU ADAPTE

- .github/workflows/release.yml
- .github/workflows/deploy-assisted.yml
- automation/src/deploy-assessment.mjs
- automation/src/deployment-manifest.mjs
- automation/src/rollback-plan.mjs
- testes e schemas necessários

Não crie um segundo agente. Reutilize classificação, sanitização, fallback, renderização e política já existentes.

RELEASE

O workflow release.yml deve executar em tags v* e:

1. fazer checkout da tag;
2. configurar Node 22;
3. executar npm ci;
4. executar lint, testes e build;
5. gerar release-manifest.json;
6. gerar release-notes.md;
7. empacotar frontend/dist e arquivos necessários do backend;
8. publicar copa-figurinhas-\${VERSION}.tgz e os manifests como artefatos.

O release-manifest.json deve conter:

- application
- version
- commitSha
- generatedAt
- gateResults
- artifact

- deployableTo
- productionRequiresApproval

DEPLOY ASSISTIDO

O workflow `deploy-assisted.yml` deve usar `workflow_dispatch` com inputs:

- `environment`: `staging` ou `production`
- `action`: `deploy` ou `rollback`
- `releaseVersion`: `branch`, `tag` ou `SHA`

JOBS

1. `assess`
2. `deploy-staging`
3. `deploy-production`
4. `rollback-plan`

ASSESS

O job `assess` deve:

- fazer `checkout` da `releaseVersion`;
- executar `npm ci`;
- executar `lint`, testes e `build` preservando logs;
- chamar o Pipeline Guardian;
- gerar `deploy-assessment.json` e `deploy-assessment.md`;
- publicar o Markdown no `GITHUB_STEP_SUMMARY`;
- publicar logs e `assessment` como artefatos;
- expor `agentRecommendation`, `policyDecision` e `requiresHumanApproval` como outputs.

CONTRATO DA AVALIAÇÃO

Inclua:

- `assessmentId`
- `releaseVersion`
- `targetEnvironment`
- `action`
- `gateResults`
- `summary`
- `signal`
- `probableCause`
- `evidence`
- `riskLevel`
- `confidence`
- `nextSteps`
- `agentRecommendation`
- `policyDecision`
- `requiresHumanApproval`
- `limitations`
- `usedFallback`
- `generatedAt`

REGRAS DA POLÍTICA

- `lint` falhou: `blocked`
- testes falharam: `blocked`
- `build` falhou: `blocked`
- segurança detectada: `blocked` e risco `high`
- confiança `low`: `blocked`
- contexto insuficiente: `blocked`
- `staging` com gates aprovados: `eligible_for_staging`
- `production` com gates aprovados: `requires_human_approval`
- `rollback` de `production`: `requires_human_approval`

A política deve prevalecer sobre qualquer recomendação insegura do modelo.

DEPLOY STAGING

Executar somente quando:

- environment == staging
- action == deploy
- policyDecision == eligible_for_staging

O job deve usar environment: staging e gerar deployment-manifest.json com:

- application
- releaseVersion
- environment
- action
- status: simulated
- commitSha
- policyDecision
- requiresHumanApproval: false
- executedAt

DEPLOY PRODUCTION

Executar somente quando:

- environment == production
- action == deploy
- policyDecision == requires_human_approval

O job deve usar environment: production. Não contorne protection rules.

Depois da aprovação, gere apenas um deployment-manifest.json com status simulated.

Se a aprovação obrigatória não estiver disponível, encerre após o assessment e informe claramente que a proteção externa não está configurada.

ROLLBACK

Quando action == rollback, gere rollback-plan.json com:

- currentVersion
- targetVersion
- environment
- probableCause
- evidence
- risks
- requiredValidations
- recommendedSteps
- requiresHumanApproval
- status: plan_only

Não execute rollback real.

SEGURANÇA

- permissões mínimas;
- ferramentas somente de leitura durante a análise;
- não imprimir secrets;
- não permitir que a LLM modifique policyDecision;
- não criar commit, push, PR ou merge;
- não acessar infraestrutura externa.

FALLBACK

Sem OPENAI_API_KEY ou OPENAI_MODEL:

- usar o caminho determinístico existente;
- manter o mesmo contrato;
- marcar usedFallback como true;
- aplicar a mesma política.

TESTES

Crie testes para:

- staging com gates verdes;
- staging com testes falhando;
- production com gates verdes;
- production sempre exigindo aprovação;
- risco alto;
- confiança baixa;
- política prevalecendo;
- rollback de staging;
- rollback de production;
- geração dos manifests e do plano.

VALIDAÇÃO

Execute:

- npm run lint
- npm run test
- npm run build
- npm run ci

Verifique a sintaxe dos YAMLS e mostre exemplos dos arquivos gerados.

Não faça commit nem push.

14.5 Validar localmente

```
npm run lint
npm run test
npm run build
npm run ci
```

```
git status
git diff --stat
git diff
```

Resultado esperado

A main continua saudável. Os testes do workspace automation validam staging, produção, bloqueios e rollback. Os workflows não executam nenhuma ação externa durante a validação local.

14.6 Versionar e abrir a Pull Request

```
git add .github/workflows/release.yml .github/workflows/deploy-assisted.yml automation docs package.json
package-lock.json
```

```
git commit -m "ci: add release and assisted deployment workflows"
git push -u origin feat/release-and-deploy
```

Abra a Pull Request com a comparação abaixo:

```
base: main
compare: feat/release-and-deploy
```

Título recomendado:

```
ci: add release and assisted deployment workflows
```

Use Create pull request. Esta é uma implementação real e deve ser mesclada depois que o CI estiver verde. Não use Draft, salvo se o trabalho ainda estiver incompleto.

Descrição resumida da PR

Adiciona release versionada, avaliação de prontidão, deploy simulado para staging, aprovação obrigatória para production e geração de plano de rollback. A recomendação do agente permanece separada da decisão da política.

14.7 Criar os environments no GitHub

1. Acesse Settings > Environments.
2. Crie o environment staging.
3. Crie o environment production.
4. Em production, configure Required reviewers e Prevent self-review quando o recurso estiver disponível.
5. Não armazene a OPENAI_API_KEY no código. Use Secrets and variables > Actions ou secrets específicos do environment.

Alternativa didática

Quando a aprovação obrigatória não estiver disponível, o workflow deve parar após gerar o assessment de production. Não simule uma autorização automática apenas para completar a execução.

14.8 Criar a primeira release

```
git switch main
git pull --ff-only origin main

git tag -a v0.1.0 -m "CopaFigurinhas v0.1.0"
git push origin v0.1.0
```

O workflow de release deve gerar o pacote, o manifesto e as notas da versão. Verifique a execução em Actions > Release e baixe os artefatos.

Artefato	Finalidade
copa-figurinhas-v0.1.0.tgz	Pacote da versão que seria promovida.
release-manifest.json	Rastreia versão, commit, gates e artefato.
release-notes.md	Resume as informações da release para leitura humana.

14.9 Executar o deploy simulado em staging

6. Acesse Actions > Deploy Assisted > Run workflow.
7. Selecione a branch main para executar o workflow.
8. Informe environment: staging.
9. Informe action: deploy.
10. Informe releaseVersion: v0.1.0.
11. Execute e abra o Job Summary do job assess.

```
{
  "agentRecommendation": "eligible_for_staging",
  "policyDecision": "eligible_for_staging",
```

```
"requiresHumanApproval": false
}
```

Depois, abra o job de staging e baixe o artefato deployment-manifest-staging. O manifesto esperado registra environment como staging e status como simulated.

14.10 Avaliar production

12. Execute novamente o workflow.
13. Informe environment: production.
14. Informe action: deploy.
15. Informe releaseVersion: v0.1.0.
16. Observe a diferença entre a recomendação técnica e a autorização final.

```
{
  "agentRecommendation": "technically_ready",
  "policyDecision": "requires_human_approval",
  "requiresHumanApproval": true
}
```

Interpretação

O agente considera a versão tecnicamente pronta. A política não autoriza produção automaticamente. Quando o environment possui proteção, o job aguarda aprovação humana.

14.11 Gerar um plano de rollback

17. Abra Actions > Deploy Assisted > Run workflow.
18. Escolha action: rollback.
19. Informe o environment e a versão alvo.
20. Execute o workflow.
21. Abra o artefato rollback-plan e revise riscos, validações e passos recomendados.

Limite da prática

O rollback é apenas planejado. Em production, a política sempre exige aprovação humana. O agente não executa comandos de reversão.

14.12 Como interpretar a prática

Resultado	Interpretação
Assess aprovado + staging executado	Os gates passaram e a política permitiu uma promoção simulada.
Assess aprovado + production aguardando	A versão está tecnicamente pronta, mas precisa de aprovação.
Assess bloqueado	Um gate falhou, o risco é alto ou o contexto é insuficiente.
Rollback plan_only	O sistema produziu um plano auditável, sem executar a reversão.

15. Slide 15 - Classificação de falhas e sugestão de causa

Exemplo: erro intencional na regra duplicateCopies

Nesta demonstração, o código continua sintaticamente válido e o build continua funcionando, mas uma regra de negócio passa a produzir valores incorretos. O cenário permite observar como o agente conecta a falha do teste, o diff da Pull Request e a regra alterada.

15.1 Regra correta

A primeira cópia de uma figurinha pertence ao álbum. Somente as cópias adicionais são repetidas.

```
export function duplicateCopies(quantity) {
  return Math.max(quantity - 1, 0);
}
```

Quantidade total	Repetidas corretas
0	0
1	0
2	1
3	2

15.2 Criar a branch da demonstração

```
git switch main
git pull --ff-only origin main
git switch -c demo/slide-15-16-test-diagnosis
```

Importante
Esta branch conterá uma falha intencional. Ela deve ser usada apenas em uma Draft Pull Request e nunca deve ser mesclada na main.

15.3 Prompt para introduzir a falha

Como usar
Cole o prompt abaixo no Claude Code depois de confirmar a branch indicada. O modelo deve analisar o repositório antes de alterar arquivos e não deve executar commit ou push automaticamente.

Você é um especialista em JavaScript, testes automatizados e análise de regras de negócio.

Trabalhe no repositório atual da aplicação CopaFigurinhas.

Antes de alterar arquivos:

1. confirme que a branch atual é demo/slide-15-16-test-diagnosis;
2. localize a função duplicateCopies;
3. localize os testes que validam essa regra;
4. não altere os testes;

5. apresente a alteração que será feita;
6. não faça commit;
7. não faça push.

OBJETIVO

Introduzir uma única falha intencional para demonstrar a análise do Pipeline Guardian.

ALTERAÇÃO

Substitua o retorno correto:

`Math.max(quantity - 1, 0)`

pelo retorno incorreto:

`quantity`

A alteração deve fazer com que a primeira cópia também seja contabilizada como repetida.

RESTRIÇÕES

- altere somente a implementação de `duplicateCopies`;
- não altere testes;
- não introduza erro de sintaxe;
- não introduza erro de lint;
- não introduza erro de build;
- não corrija a falha depois de criá-la;
- não faça commit;
- não faça push.

VALIDAÇÃO

Execute:

- os testes relacionados ao relatório;
- `npm run lint`;
- `npm run build`.

Ao final, informe:

- arquivo alterado;
- trecho correto anterior;
- trecho incorreto atual;
- testes que falharam;
- valores esperados e recebidos;
- impacto da falha no relatório de figurinhas;
- resultado de lint, testes e build.

15.4 Validar o cenário localmente

```
npm run lint
npm run test
npm run build
```

Etapa	Resultado esperado	Por quê
Lint	Aprovado	O código é válido e não viola as regras estáticas.
Testes	Reprovados	A regra de negócio retorna um valor incorreto.

Etapa	Resultado esperado	Por quê
Build	Aprovado	A aplicação ainda pode ser compilada.

15.5 Commit, push e Draft Pull Request

```
git status
git diff -- backend/src/services/report.js

git add backend/src/services/report.js
git commit -m "demo: introduce duplicate calculation defect"
git push -u origin demo/slide-15-16-test-diagnosis
```

Abra a comparação:

```
base: main
compare: demo/slide-15-16-test-diagnosis
```

Título da PR:

```
demo: Pipeline Guardian analyzes business rule failure
```

Use Create draft pull request. Não use uma PR normal e não faça merge.

Descrição recomendada

Demonstração educacional

Esta Draft Pull Request contém uma falha intencional na regra de cálculo de figurinhas repetidas.

Objetivo

Demonstrar como o Pipeline Guardian:

- recebe o resultado dos testes;
- analisa o log técnico;
- consulta as alterações da Pull Request;
- classifica a falha;
- sugere uma causa provável;
- avalia impacto e confiança;
- recomenda o próximo passo;
- produz uma resposta estruturada;
- bloqueia a continuidade por meio do CI Gate.

Resultado esperado

- Lint: aprovado
- Testes: reprovados
- Build: aprovado
- Diagnose: aprovado
- CI Gate: reprovado

Atenção

Falha criada exclusivamente para fins educacionais.

****DO NOT MERGE****

15.6 Acompanhar o pipeline

Na Draft PR, abra Checks ou acesse Actions > CI. O resultado esperado é:

Job	Status esperado	Significado
Quality	Sucesso	A falha não é de estilo ou sintaxe.
Tests	Falha	A regra de negócio não atende ao comportamento esperado.
Build	Sucesso	O pacote ainda pode ser gerado.
Diagnose	Sucesso	O agente conseguiu analisar a execução.
CI Gate	Falha	Código defeituoso não pode avançar.

Conceito central

Diagnose aprovado não significa pipeline aprovado. O agente pode analisar corretamente uma execução que continua bloqueada por uma falha real.

15.7 O caminho de investigação esperado

1. inspect_job_results
→ apenas Tests falhou
2. read_pipeline_log("test")
→ AssertionError com esperado e recebido
3. read_pull_request_diff
→ duplicateCopies foi alterado
4. read_source_excerpt
→ a primeira cópia deixou de ser descontada
5. finish
→ confiança alta e decisão blocked

15.8 O que observar no comentário do agente

- Sinal: a mensagem objetiva extraída do teste.
- Tipo de falha: test.
- Causa provável: a primeira cópia está sendo contabilizada como repetida.
- Evidências: log, diff e trecho do código.
- Impacto: relatório incorreto para o usuário.
- Confiança: força da hipótese.
- Próximo passo: restaurar o cálculo e reexecutar o pipeline.
- Decisão: blocked.

15.9 Reexecutar sem alterar o código

A Draft PR pode permanecer aberta para novas turmas. Para criar uma execução com horário atual, use um commit vazio:

```
git switch demo/slide-15-16-test-diagnosis
git pull --ff-only origin demo/slide-15-16-test-diagnosis

git commit --allow-empty -m "demo: rerun Slides 15 and 16 pipeline diagnosis"

git push origin demo/slide-15-16-test-diagnosis
```

O que o commit vazio faz

Ele cria um novo SHA e dispara o workflow, mas não modifica nenhum arquivo. A falha intencional continua isolada na branch demo.

16. Slide 16 - Saída estruturada para o desenvolvedor

Do log bruto a uma resposta clara, rastreável e acionável

O slide 16 utiliza a mesma execução do slide 15. Não é necessário criar outra falha. O objetivo agora é interpretar como o agente organiza o diagnóstico e como essa saída pode ser reutilizada pelo GitHub, por um dashboard ou por um canal ChatOps.

16.1 Onde visualizar

22. Abra a Draft Pull Request da demonstração.
23. Na aba Conversation, localize o comentário Pipeline Guardian.
24. Na aba Checks, abra Diagnose (Pipeline Guardian).
25. No Summary, leia a versão Markdown.
26. No final da execução, baixe o artefato pipeline-guardian-diagnosis.
27. Abra diagnosis.json e diagnosis.md.

16.2 Estrutura da resposta

Campo	Pergunta respondida
summary	O que aconteceu no pipeline?
signal	Qual fato técnico foi observado?
failureType	Em qual categoria a falha foi classificada?
probableCause	Qual hipótese explica o sinal?
evidence	Quais fontes sustentam a hipótese?
impact	Qual comportamento ou entrega foi afetado?
confidence	Quão forte é a conclusão?
nextSteps	O que deve ser validado ou corrigido?
agentRecommendation	O que o agente recomenda?
policyDecision	O que a governança autoriza?

Campo	Pergunta respondida
limitations	O que não foi possível afirmar ou executar?

16.3 Exemplo de diagnosis.json

```
{
  "goal": "ci_diagnosis",
  "pipelineStatus": "failed",
  "failureType": "test",
  "signal": "AssertionError: expected 3 to be 2",
  "probableCause": "A primeira cópia foi contabilizada como repetida.",
  "evidence": [
    {
      "source": "tests.log",
      "excerpt": "expected 3 to be 2"
    },
    {
      "source": "pr.diff",
      "excerpt": "return quantity"
    }
  ],
  "impact": "O relatório informa repetidas em quantidade incorreta.",
  "riskLevel": "medium",
  "confidence": "high",
  "toolsUsed": [
    "inspect_job_results",
    "read_pipeline_log",
    "read_pull_request_diff",
    "read_source_excerpt"
  ],
  "stopReason": "sufficient_evidence",
  "agentRecommendation": "blocked",
  "policyDecision": "blocked",
  "requiresHumanApproval": false,
  "limitations": []
}
```

16.4 Evidência não é hipótese

Tipo	Exemplo
Evidência	AssertionError: expected 3 to be 2.
Evidência	O diff alterou Math.max(quantity - 1, 0) para quantity.
Hipótese	A primeira cópia passou a ser contada como repetida.
Decisão	A política mantém o pipeline bloqueado.

Ponto de atenção

A causa provável deve ser apresentada como hipótese fundamentada. O agente não deve inventar arquivos, comandos ou certezas que não estejam sustentadas pelas evidências.

16.5 Markdown e JSON têm papéis diferentes

Formato	Uso principal
diagnosis.md	Leitura humana: comentário na PR, Job Summary ou mensagem ChatOps.
diagnosis.json	Integração: dashboards, APIs, banco de dados, automações e formatadores de canal.

16.6 Checklist de análise

- O sinal foi extraído do log sem códigos ANSI ou ruído desnecessário.
- A causa provável está ligada às evidências da execução.
- O impacto descreve a consequência para a aplicação ou para a entrega.
- Os próximos passos são específicos e executáveis.
- A confiança é compatível com a quantidade e qualidade das evidências.
- A recomendação do agente está separada da decisão da política.
- A saída não expõe tokens, secrets ou dados sensíveis.
- O agente não corrige, não aprova e não faz merge sozinho.

16.7 Encerrar a demonstração

```
git switch main
git pull --ff-only origin main

npm run lint
npm run test
npm run build
```

A main deve continuar saudável. A Draft PR permanece aberta e a falha continua isolada na branch demo. Não approve e não faça merge dessa PR.

17. Slide 19 - Demonstração guiada: agente analisador de pipeline

Notebook para acompanhar o raciocínio, as ferramentas e a decisão

O desafio final não deve criar um segundo agente. O notebook funcionará como uma camada didática que prepara cenários, chama o Pipeline Guardian existente e exibe o caminho da investigação. Assim, o aluno observa o estado inicial, as ferramentas escolhidas, as evidências coletadas, o critério de parada e a decisão da política.

17.1 Objetivo do notebook

- Executar cenários controlados sem depender de uma Pull Request real.
- Reutilizar o agente e as ferramentas já implementados no workspace automation.
- Mostrar a diferença entre resultado do workflow, recomendação do agente e decisão da política.
- Exibir investigationTrace, toolsUsed, confidence e stopReason.
- Permitir que o aluno altere um cenário e observe uma nova rota de investigação.

17.2 Arquitetura da demonstração

```

Notebook Python
  ↓ executa subprocess
Script Node de demonstração
  ↓ reutiliza
Pipeline Guardian existente
  ↔ ferramentas somente de leitura
  ↓
JSON estruturado
  ↓
Notebook apresenta trace, evidências e decisão

```

Por que usar um script Node intermediário?

O projeto e o agente foram implementados em JavaScript. O notebook Python chama um CLI Node e lê o JSON produzido, evitando duplicar a lógica do agente em outra linguagem.

17.3 Estrutura esperada

```

notebooks/
├── slide-19-agent-analisador-pipeline.ipynb

automation/src/
├── guided-demo.mjs

automation/fixtures/guided-demo/
├── success/
├── test-failure/
├── build-failure/
├── security/

reports/guided-demo/
├── result.json
├── result.md

```

17.4 Prompt completo - Demonstração guiada do Slide 19

Como usar

Cole o prompt abaixo no Claude Code depois de confirmar a branch indicada. O modelo deve analisar o repositório antes de alterar arquivos e não deve executar commit ou push automaticamente.

Você é um especialista em agentes de IA, Node.js, Jupyter Notebook, CI/CD, segurança, observabilidade e didática para desenvolvedores.

Trabalhe no repositório atual pipeline-guardian-agent-demo.

Antes de alterar arquivos:

1. confirme que a branch atual é feat/slide-19-guided-demo;
2. inspecione o workspace automation;
3. localize o estado da investigação, o planner, o investigation loop, o tool registry, a política e os renderizadores;
4. identifique a API pública existente para executar o Pipeline Guardian;
5. apresente um plano curto;
6. implemente somente depois do plano;
7. não duplique o agente;
8. não altere a aplicação CopaFigurinhas;
9. não faça commit;

10. não faça push;
11. não implemente deploy real;
12. não use LangGraph.

OBJETIVO

Criar a demonstração guiada do Slide 19 em um notebook Jupyter.

O notebook deve permitir que um aluno execute cenários controlados e observe:

- estado inicial do pipeline;
- objetivo da investigação;
- ferramentas disponíveis;
- ferramentas selecionadas;
- observações produzidas;
- evidências adicionadas;
- hypotheses;
- investigationTrace;
- confidence;
- stopReason;
- agentRecommendation;
- policyDecision;
- limitations;
- usedFallback ou plannerMode.

LINHAGEM

O notebook não pode reimplementar o classificador, o planner, as ferramentas ou a política. Ele deve chamar o Pipeline Guardian já existente.

Como o agente é JavaScript e o notebook usará kernel Python, crie um script CLI:

automation/src/guided-demo.mjs

O notebook deve chamar esse script com subprocess e ler JSON pelo stdout ou por um arquivo de relatório.

ARQUIVOS

Crie:

- notebooks/slide-19-agent-analisador-pipeline.ipynb
- automation/src/guided-demo.mjs
- automation/fixtures/guided-demo/success/
- automation/fixtures/guided-demo/test-failure/
- automation/fixtures/guided-demo/build-failure/
- automation/fixtures/guided-demo/security/
- testes necessários

Não remova fixtures existentes.

CLI

O script guided-demo.mjs deve aceitar:

node automation/src/guided-demo.mjs --scenario <nome> --output <arquivo>

Cenários aceitos:

- success
- test-failure
- build-failure
- security

O CLI deve:

1. validar o cenário;

2. montar o estado inicial;
3. apontar para os artefatos do cenário;
4. chamar o investigation loop existente;
5. aplicar a política existente;
6. produzir JSON válido;
7. salvar reports/guided-demo/result.json;
8. salvar reports/guided-demo/result.md;
9. imprimir no stdout apenas um resumo JSON seguro;
10. retornar exit code 0 quando a demonstração foi processada, mesmo quando a policyDecision for blocked;
11. retornar exit code diferente de zero somente em erro técnico da demonstração.

CENÁRIO SUCCESS

Metadados:

- quality: success
- tests: success
- build: success
- environment: staging

Resultado esperado:

- pipelineStatus: success
- agentRecommendation: eligible_for_staging
- policyDecision: eligible_for_staging
- requiresHumanApproval: false

O agente deve evitar ler logs desnecessários quando os gates já são suficientes.

CENÁRIO TEST-FAILURE

Use a regra duplicateCopies como contexto.

Artefatos devem incluir:

- tests.log com AssertionError;
- pr.diff mostrando a mudança de Math.max(quantity - 1, 0) para quantity;
- trecho seguro do arquivo report.js.

Caminho esperado:

```
inspect_job_results  
→ read_pipeline_log(test)  
→ read_pull_request_diff  
→ read_source_excerpt  
→ finish
```

Resultado esperado:

- failureType: test
- probableCause relacionada à primeira cópia contada como repetida
- confidence: high
- agentRecommendation: blocked
- policyDecision: blocked

CENÁRIO BUILD-FAILURE

Artefatos devem incluir:

- build.log com Could not resolve ./components/StickerBadge.jsx;
- pr.diff com o novo import;
- arquivo inexistente.

Caminho esperado:

```
inspect_job_results  
→ read_pipeline_log(build)  
→ read_pull_request_diff  
→ check_path_exists  
→ finish
```

Resultado esperado:

- failureType: build
- probableCause: componente local inexistente ou caminho incorreto
- policyDecision: blocked

CENÁRIO SECURITY

Use somente tokens fictícios.

Artefatos devem conter um padrão de segredo que possa ser detectado e mascarado.

Caminho esperado:

- inspect_job_results
- scan_sensitive_data
- read_pipeline_log mascarado, somente se necessário
- finish

Resultado esperado:

- failureType: security
- riskLevel: high
- policyDecision: blocked
- nenhum segredo original no JSON, Markdown ou stdout

NOTEBOOK

O notebook deve ser didático e possuir células Markdown e código nesta ordem:

1. Título e objetivo
2. Arquitetura da demonstração
3. Diferença entre workflow, agente e política
4. Verificação do ambiente
5. Função Python run_scenario(scenario)
6. Cenário success
7. Estado inicial do cenário
8. Execução do Pipeline Guardian
9. Caminho da investigação
10. Evidências coletadas
11. Saída estruturada
12. Recomendação do agente versus decisão da política
13. Cenário test-failure
14. Cenário build-failure
15. Cenário security
16. Comparação dos cenários em uma tabela
17. Exercício para o aluno
18. Checklist final

A função Python deve usar subprocess.run com:

- timeout;
- capture_output=True;
- text=True;
- verificação de erros;
- parsing do JSON;
- mensagens claras quando o Node ou um arquivo estiver ausente.

Não use shell=True.

Não imprima variáveis de ambiente.

Não exponha OPENAI_API_KEY.

APRESENTAÇÃO NO NOTEBOOK

Use Python padrão e, se já estiver disponível, pandas para tabelas.

Não adicione bibliotecas pesadas apenas para apresentação.

Crie funções para mostrar:

- resumo do cenário;

- toolsUsed;
- investigationTrace;
- evidence;
- agentRecommendation e policyDecision lado a lado;
- limitations;
- usedFallback ou plannerMode.

Quando o resultado estiver blocked, o notebook deve explicar que o agente foi executado com sucesso, mas a política impediu a continuidade.

EXERCÍCIO PARA O ALUNO

Inclua uma célula Markdown orientando o aluno a criar um quinto cenário:

environment-failure

Sinal sugerido:

missing env var: VITE_API_URL

O aluno deverá prever:

- qual ferramenta será selecionada;
- qual classificação será produzida;
- qual decisão da política será aplicada.

SEGURANÇA

- ferramentas somente de leitura;
- máximo de 5 chamadas;
- timeout máximo de 30 segundos no agente;
- timeout no subprocess do notebook;
- path traversal bloqueado;
- .env e secrets inacessíveis;
- tokens fictícios mascarados;
- nenhuma escrita na aplicação;
- nenhum comando definido pela LLM;
- nenhum deploy, merge, commit ou push.

FALLBACK

O notebook deve funcionar sem OPENAI_API_KEY.

Quando não houver modelo configurado:

- usar o planner determinístico existente;
- marcar plannerMode como deterministic ou usedFallback como true;
- manter os mesmos campos de saída.

TESTES

Crie testes para:

- cenário inválido;
- success;
- test-failure;
- build-failure;
- security;
- saída JSON válida;
- segredo mascarado;
- exit code 0 em diagnóstico blocked;
- erro técnico retornando exit code diferente de zero;
- relatórios gerados;
- notebook sem outputs antigos antes da entrega.

VALIDAÇÃO

Execute:

- npm run lint

```
- npm run test
- npm run build
- npm run ci
- node automation/src/guided-demo.mjs --scenario test-failure
- node automation/src/guided-demo.mjs --scenario success
```

Execute o notebook de forma não interativa:

```
jupyter nbconvert \
--to notebook \
--execute notebooks/slide-19-agent-analisador-pipeline.ipynb \
--output slide-19-agent-analisador-pipeline.executed.ipynb \
--ExecutePreprocessor.timeout=180
```

Ao final:

- mostre a árvore dos arquivos criados;
- informe os comandos executados;
- apresente um resumo dos quatro cenários;
- confirme que nenhum segredo foi exposto;
- não faça commit;
- não faça push.

17.5 Criar a branch antes de executar o prompt

```
git switch main
git pull --ff-only origin main
git switch -c feat/slide-19-guided-demo
```

Use uma sessão limpa no Claude Code e cole o prompt completo. O Claude deve analisar os módulos existentes e adaptar os nomes reais do projeto, sem criar uma arquitetura paralela.

17.6 Executar localmente

```
npm run lint
npm run test
npm run build
npm run ci

node automation/src/guided-demo.mjs --scenario success
node automation/src/guided-demo.mjs --scenario test-failure
node automation/src/guided-demo.mjs --scenario build-failure
node automation/src/guided-demo.mjs --scenario security

jupyter lab
```

Abra notebooks/slide-19-agent-analisador-pipeline.ipynb e execute as células na ordem. O notebook deve mostrar caminhos diferentes de investigação para cenários diferentes.

17.7 Validar o notebook sem interface

```
jupyter nbconvert --to notebook --execute notebooks/slide-19-agent-analisador-pipeline.ipynb --output
slide-19-agent-analisador-pipeline.executed.ipynb --ExecutePreprocessor.timeout=180
```

Ponto de atenção

O notebook deve retornar sucesso técnico mesmo quando a política bloqueia o cenário. Bloqueio de deploy é um resultado de negócio esperado; erro técnico é falha de execução do laboratório.

17.8 Versionar e abrir a Pull Request

```
git status
git diff --stat

git add notebooks automation docs package.json package-lock.json
git commit -m "feat: add guided Pipeline Guardian notebook"
git push -u origin feat/slide-19-guided-demo
```

Abra uma Pull Request normal:

```
base: main
compare: feat/slide-19-guided-demo
```

Título recomendado:

```
feat: add guided Pipeline Guardian notebook
```

Use Create pull request. Depois que lint, testes, build e CI Gate estiverem verdes, revise o notebook e faça o merge.

17.9 Resultado esperado

Cenário	Ferramentas principais	Decisão
success	inspect_job_results	eligible_for_staging
test-failure	job results, test log, diff, source excerpt	blocked
build-failure	job results, build log, diff, path check	blocked
security	job results, sensitive-data scan	blocked / high risk

18. Síntese final

O pipeline executa; o agente investiga; a política limita

As três práticas desta continuação representam papéis diferentes do mesmo sistema. No deploy assistido, o agente prepara uma recomendação e a política controla a promoção. Na falha de duplicateCopies, o agente interpreta um teste reprovado e relaciona o sintoma com a alteração da PR. No notebook do slide 19, o aluno acompanha o processo interno da investigação em cenários controlados.

Elemento	Responsabilidade final
Workflow	Iniciar a execução, preparar o ambiente e executar comandos determinísticos.
Agente	Escolher como investigar, reunir evidências, formular hipótese e recomendar ação.
Ferramentas	Fornecer acesso controlado a logs, diff, código, manifests e verificações.
Política	Aplicar regras fixas de segurança e autorização.

Elemento	Responsabilidade final
Humano	Aprovar ações críticas e assumir responsabilidade operacional.

Checklist de entrega

- Deploy assistido implementado sem infraestrutura real.
- Recomendação do agente separada da decisão da política.
- Production e rollback crítico condicionados à aprovação humana.
- Draft PR dos slides 15 e 16 criada com DO NOT MERGE.
- Falha duplicateCopies reproduzida com lint e build aprovados e testes reprovados.
- Comentário, Job Summary e artefatos do Pipeline Guardian revisados.
- Notebook do slide 19 executa cenários distintos e reutiliza o agente existente.
- Trace, ferramentas, evidências, confiança, stopReason e decisões estão visíveis.
- Nenhum secret é exposto.
- Código gerado pela IA foi revisado antes do merge.

Mensagem principal

A autonomia do agente está na investigação e na recomendação. A execução crítica continua limitada por política, permissões e aprovação humana.